

ROUTING ALGORITHMS

- The main function of the network layer is **routing packets from the source machine to the destination machine**. In most subnets (Datagram/Virtual Circuit) , packets will require multiple hops to make the journey
- The routing algorithm is that part of the network layer software **responsible for deciding which output line** an incoming packet should be transmitted on.
- If the **subnet uses datagrams** internally, this decision must be made a new for every arriving data packet since the best route may have changed since last time.

- If the subnet uses virtual circuits internally, routing decisions are made only when a new virtual circuit is being set up. Thereafter, data packets just follow the previously-established route. This is sometimes called session routing because a route remains in force for an entire user session.
- A router is having two processes inside it.
- One of them handles each packet as it arrives, looking up the outgoing line to use for it in the routing tables. This process is forwarding.
- The other process is responsible for filling in and updating the routing tables. That is where the routing algorithm comes into play.

- 
- Certain properties are desirable in a routing algorithm: correctness, simplicity, robustness, stability, fairness, and optimality.
 - The routing algorithm should be able to cope with changes in the topology and traffic without requiring all jobs in all hosts to be aborted and the network to be rebooted every time some router crashes.
 - Routing algorithms can be grouped into two major classes:
 - **Nonadaptive**
 - **Adaptive.**

- **Adaptive Routing algorithm**

- An adaptive routing algorithm is also known as dynamic routing algorithm.
- This algorithm makes the routing decisions based on the topology and network traffic.
- The main parameters related to this algorithm are hop count, distance and estimated transit time.

- **Non-Adaptive Routing algorithm**

- Non Adaptive routing algorithm is also known as a static routing algorithm.
- When booting up the network, the routing information stores to the routers.
- Non Adaptive routing algorithms do not take the routing decision based on the network topology or network traffic.

Basis Of Comparison	Adaptive Routing algorithm	Non-Adaptive Routing algorithm
Define	Adaptive Routing algorithm is an algorithm that constructs the routing table based on the network conditions.	The Non-Adaptive Routing algorithm is an algorithm that constructs the static table to determine which node to send the packet.
Usage	Adaptive routing algorithm is used by dynamic routing.	The Non-Adaptive Routing algorithm is used by static routing.
Routing decision	Routing decisions are made based on topology and network traffic.	Routing decisions are the static tables.
Categorization	The types of adaptive routing algorithm, are Centralized, isolation and distributed algorithm.	The types of Non Adaptive routing algorithm are flooding and random walks.
Complexity	Adaptive Routing algorithms are more complex.	Non-Adaptive Routing algorithms are simple.

•**OPTIMALITY PRINCIPLE**

- The purpose of a routing algorithm at a router is to decide which output line an incoming packet should go. The optimal path from a particular router to another may be the least cost path, the least distance path, the least time path, the least hops path or a combination of any of the above.
- The optimality principle can be logically proved as follows –
- If a better route could be found between router J and router K, the path from router I to router K via J would be updated via this route. Thus, the optimal path from J to K will again lie on the optimal path from I to K.

•**DISTANCE VECTOR ROUTING**

- The Distance vector algorithm is a dynamic algorithm.
- It is also called Bellman-Ford routing algorithm and the Ford-Fulkerson algorithm
- It is mainly used in ARPANET, and RIP.
- Each router maintains a distance table known as Vector.
- Each node receives information from one or more of its directly attached neighbors, performs calculation and then distributes the result back to its neighbors.
- Information sharing at regular intervals - Within 30 seconds, the router sends the information to the neighboring routers.

- **Distance Vector (DV) Algorithm**

- A router transmits its distance vector to each of its neighbors in a routing packet.
- Each router receives and saves the most recently received distance vector from each of its neighbors.
- A router recalculates its distance vector when:
 - It receives a distance vector from a neighbor containing different information than before.
 - It discovers that a link to a neighbor has gone down.
- The DV calculation is based on minimizing the cost to each destination
- From time-to-time, each node sends its own distance vector estimate to neighbors.
- When a node x receives new DV estimate from any neighbor v , it saves v 's distance vector and it updates its own DV using B-F equation:
 - **$Dx(y) = \min \{ C(x,v) + Dv(y), Dx(y) \}$ for each node $y \in N$**

Key Points about Distance Vector Routing

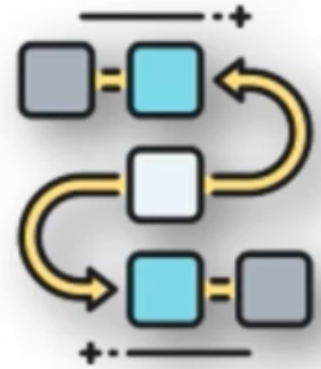


Information about the network

Each router is designed to share vector data to each of its nodes throughout the network.

Routing Pattern in the network

The data shared by the routers is transmitted only to those nodes with those it has direct link.



Sharing data periodically

The nodes share vector data at regular interval in the network.

Algorithm Applied

Bellman-Ford Algorithm

$$d_x(y) = \min_v \{c(x, v) + d_v(y)\}$$

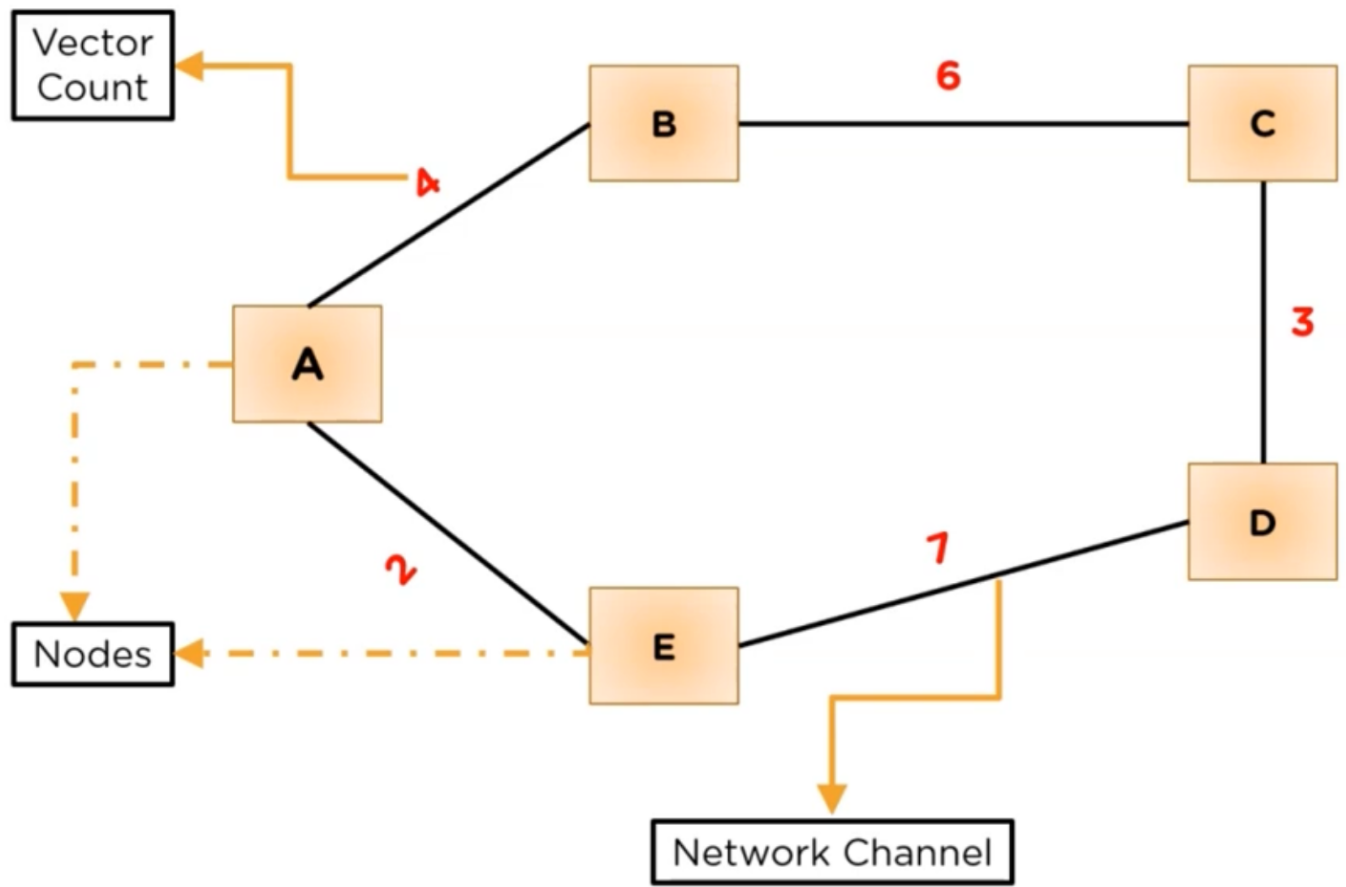
Where,

$d_x(y)$ - The least distance from x to y .

$c(x, v)$ - Node x 's cost from each of its neighbour v .

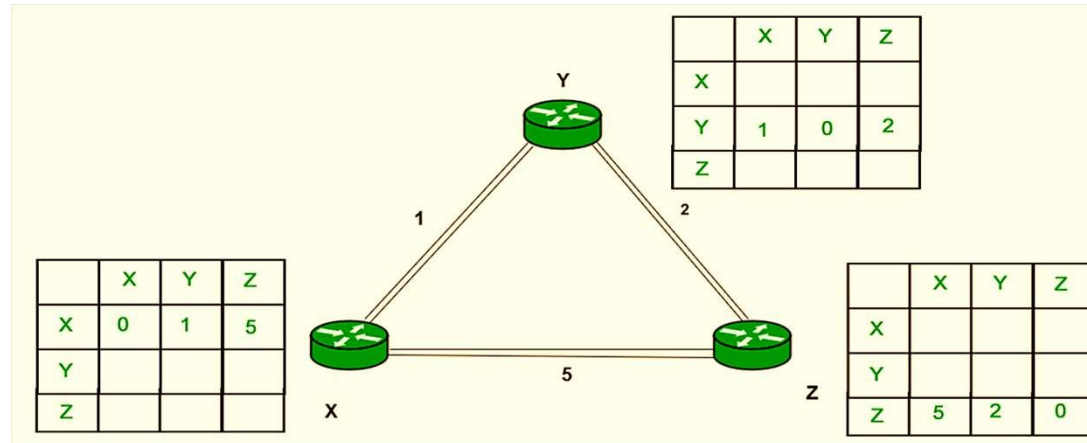
$d_v(y)$ - Distance of each neighbor from initial node.

$\min_v$ - Selecting the minimum distance for the data packet.



- Example**

- Consider 3-routers X, Y and Z as shown in figure. Each router have their routing table. Every routing table will contain distance to the destination nodes.

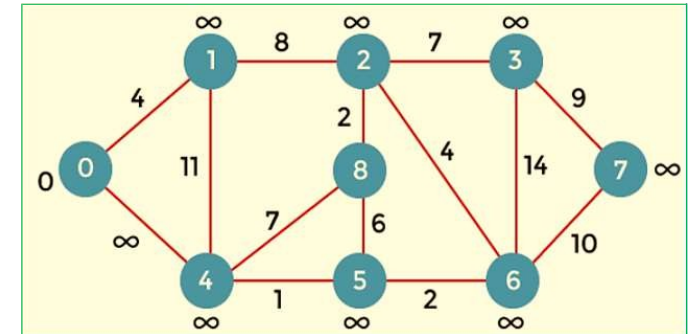
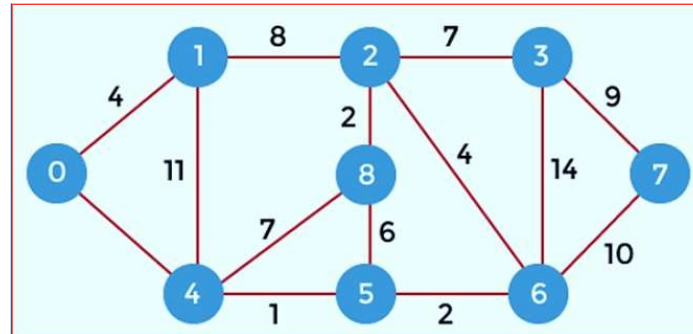


•**SHORTEST PATH ROUTING**

- It is one of the simple static routing algorithms that are widely used for routing in the network.
- The basic idea of it is to build a graph with each node representing a router and each line representing a communication link.
- To choose a route between any two nodes in the graph the algorithm simply finds the shortest path between the nodes.
- Shortest Path means that the path in which anyone or more metrics is minimized. The metric may be distance, bandwidth, average traffic, communication cost, mean queue length, measured delay or any other factor

- **Common Shortest Path Algorithms**
- **Bellman Ford's Algorithm**
- **Dijkstra's Algorithm**
- **Floyd Warshall's Algorithm**
 - **Dijkstra's Algorithm**
- Dijkstra's algorithm is a single-source shortest path algorithm.
- Here, single-source means that only one source is given, and we have to find the shortest path from the source to all the nodes.
- Let's understand the working of Dijkstra's algorithm. Consider the below graph.

- First, we have to consider any vertex as a source vertex.
- Here we assume that 0 as a source vertex, and distance to all the other vertices is infinity.
- Initially, we do not know the distances. First, we will find out the vertices which are directly connected to the vertex 0.
- As we can observe in the below graph that two vertices are directly connected to vertex 0.



❖ The formula for calculating the distance between the vertices: **if($d(u) + c(u, v) < d(v)$) Then $d(v) = d(u) + c(u, v)$**

- Let's assume that the vertex 0 is represented by 'x' and the vertex 1 is represented by 'y'. The distance between the vertices can be calculated by using the below formula:

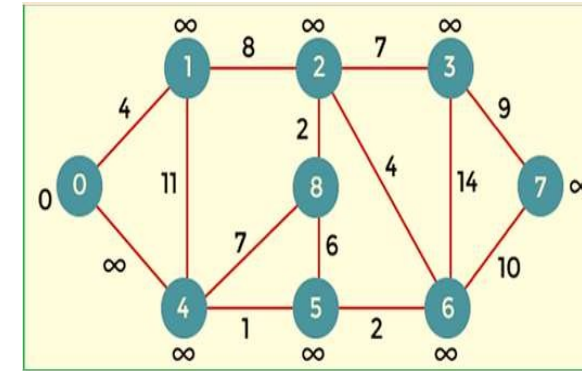
$$d(x, y) = d(x) + c(x, y) < d(y)$$

$$= (0 + 4) < \infty$$

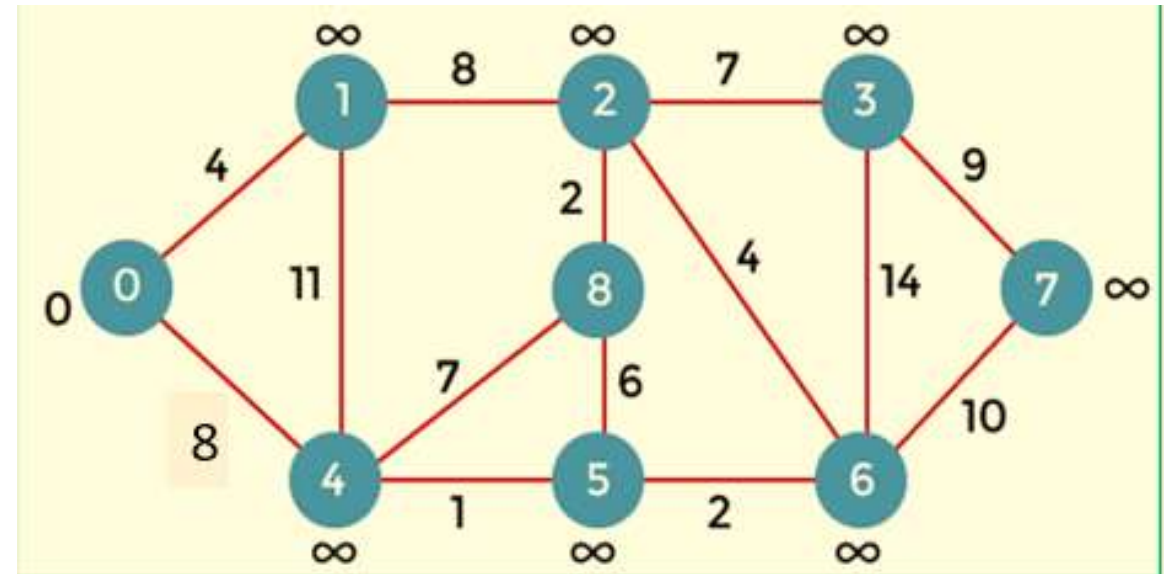
$$= 4 < \infty$$

from ∞ to 4.

Since $4 < \infty$ so we will update $d(y)$

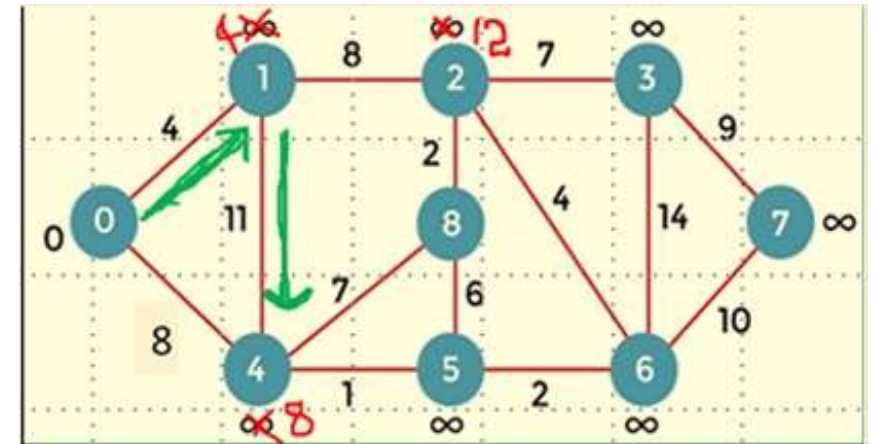
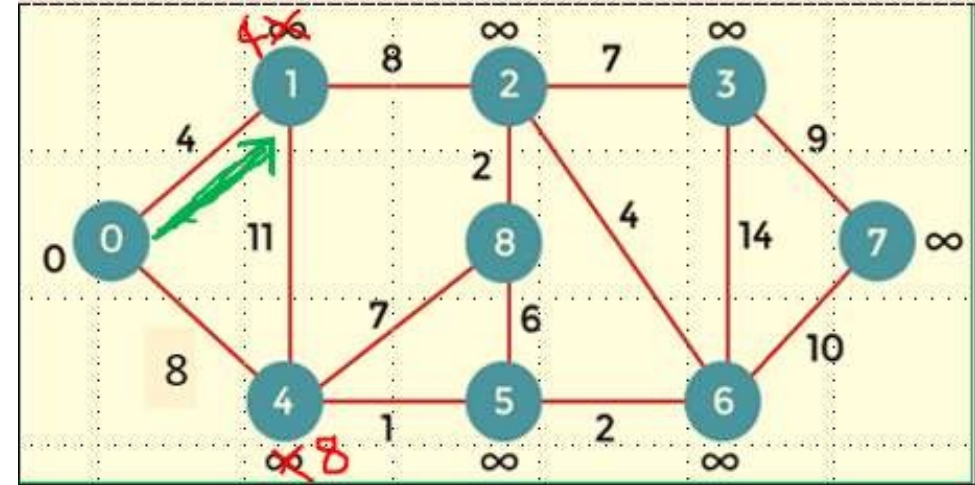


-
- Now we consider vertex 0 same as 'x' and vertex 4 as 'y'. $d(x, y) = d(x) + c(x, y) < d(y)$
 - $= (0 + 8) < \infty$
 - $= 8 < \infty$
 - Therefore, the value of $d(y)$ is 8. We replace the infinity value of vertices 1 and 4 with the values 4 and 8 respectively.
 - Now, we have found the shortest path from the vertex 0 to 1 and 0 to 4. Therefore, vertex 0 is selected. Now, we will compare all the vertices except the vertex 0.
 - Since vertex 1 has the lowest value, i.e., 4; therefore, vertex 1 is selected.



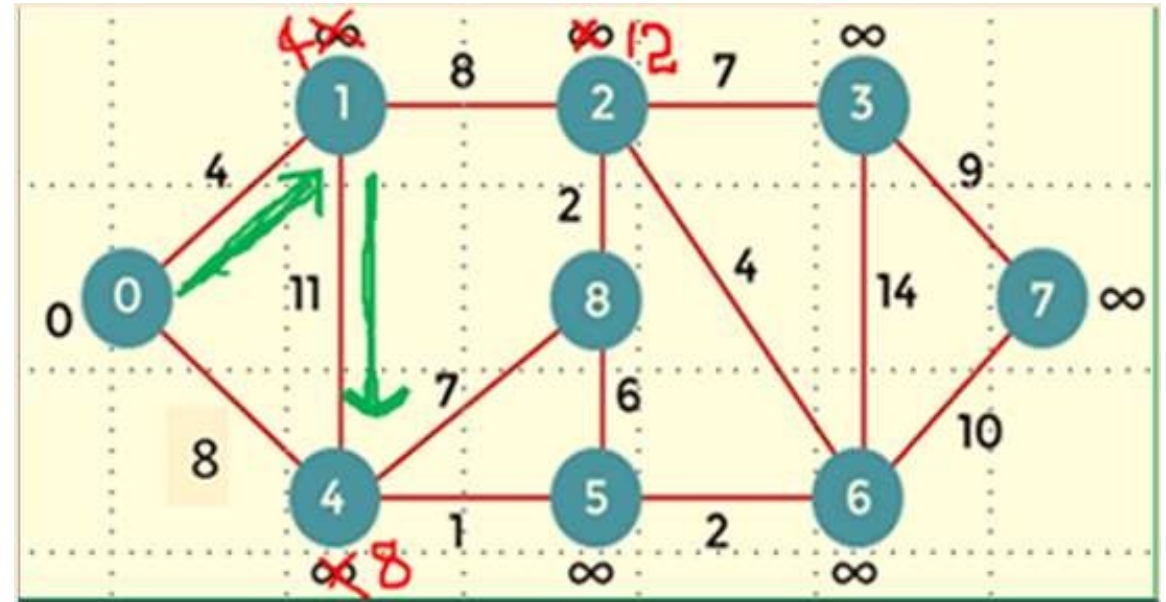
-
- Till now, two nodes have been selected, i.e., 0 and 1.
 - Now we have to compare the nodes except the node 0 and 1.
 - The node 4 has the minimum distance, i.e., 8. Therefore, vertex 4 is selected.
 - Since vertex 4 is selected, so we will consider all the direct paths from the vertex 4.
 - The direct paths from vertex 4 are 4 to 0, 4 to 1, 4 to 8, and 4 to 5.
 - Since the vertices 0 and 1 have already been selected so we will not consider the vertices 0 and 1.
 - We will consider only two vertices, i.e., 8 and 5.

-
- Since vertex 1 is selected, so we consider the path from 1 to 2, and 1 to 4.
 - First, we calculate the distance between the vertex 1 and 2. Consider the vertex 1 as 'x', and the vertex 2 as 'y'.
 - $d(x, y) = d(x) + c(x, y) < d(y)$
 - $= (4 + 8) < \infty$
 - $= 12 < \infty$
 - Since $12 < \infty$ so we will update $d(2)$ from ∞ to 12.
 - Now, we calculate the distance between the vertex 1 and vertex 4. Consider the vertex 1 as 'x' and the vertex 4 as 'y'.
 - $d(x, y) = d(x) + c(x, y) < d(y)$
 - $= (4 + 11) < 8$
 - $= 15 < 8$
 - Since 15 is not less than 8, we will not update the value $d(4)$ from 8 to 15.

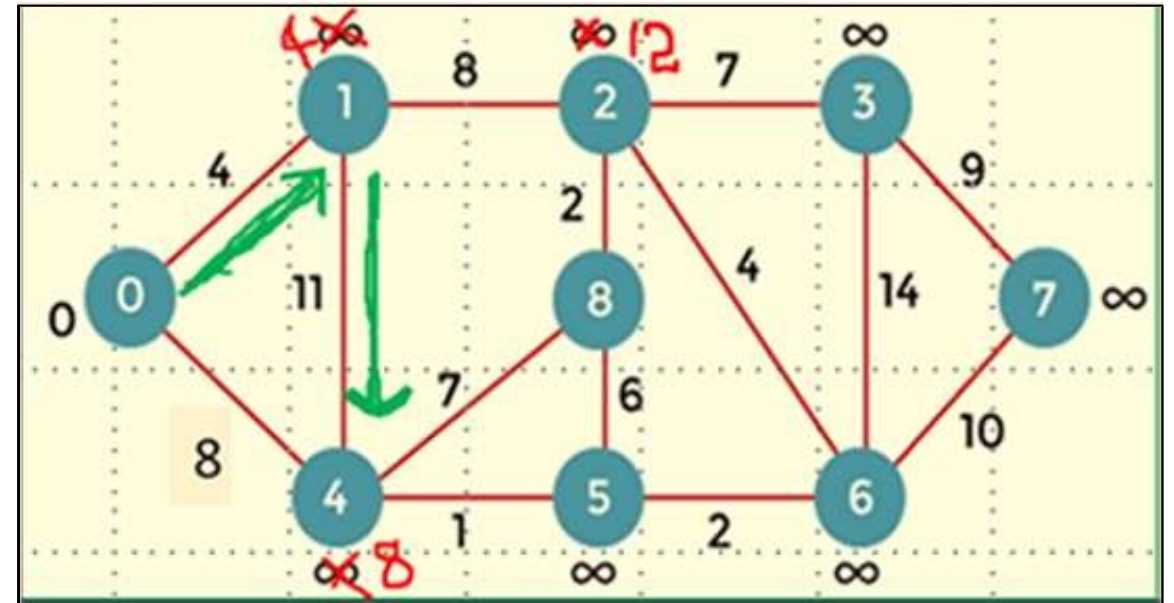


-
- Now, we consider the vertex 5. First, we calculate the distance between the vertex 4 and 5.
 - Consider the vertex 4 as 'x', and the vertex 5 as 'y'. $d(x, y) = d(x)$
+ $c(x, y) < d(y)$
 - $= (8 + 1) < \infty$
 - $= 9 < \infty$
 - Since 9 is less than the infinity, we update $d(5)$ from infinity to 9.
 - The node 5 has the minimum value, i.e., 9. Therefore, vertex 5 is selected.

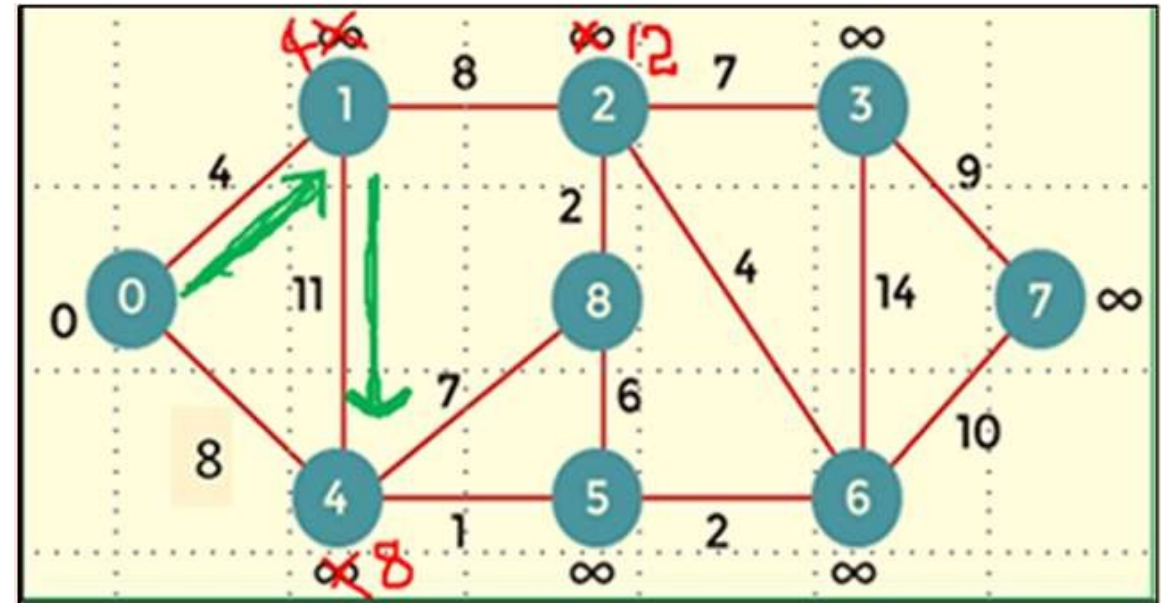
-
- Till now, two nodes have been selected, i.e., 0 and 1.
 - Now we have to compare the nodes except the node 0 and 1.
 - The node 4 has the minimum distance, i.e., 8. Therefore, vertex 4 is selected.
 - Since vertex 4 is selected, so we will consider all the direct paths from the vertex 4.
 - The direct paths from vertex 4 are 4 to 0, 4 to 1, 4 to 8, and 4 to 5.
 - Since the vertices 0 and 1 have already been selected so we will not consider the vertices 0 and 1.
 - We will consider only two vertices, i.e., 8 and 5.



-
- First, we consider the vertex 8. First, we calculate the distance between the vertex 4 and 8.
 - Consider the vertex 4 as 'x', and the vertex 8 as 'y'.
 $d(x, y) = d(x) + c(x, y) < d(y)$
 - $= (8 + 7) < \infty$
 - $= 15 < \infty$
 - Since 15 is less than the infinity so we update $d(8)$ from infinity to 15.



-
- Now, we consider the vertex 5. First, we calculate the distance between the vertex 4 and 5.
 - Consider the vertex 4 as 'x', and the vertex 5 as 'y'. $d(x, y) = d(x) + c(x, y) < d(y)$
 - $= (8 + 1) < \infty$
 - $= 9 < \infty$
 - Since 9 is less than the infinity, we update $d(5)$ from infinity to 9.
 - The node 5 has the minimum value, i.e., 9. Therefore, vertex 5 is selected.



-
- Since the vertex 5 is selected, so we will consider all the direct paths from vertex 5. The direct paths from vertex 5 are 5 to 8, and 5 to 6.
 - First, we consider the vertex 8. First, we calculate the distance between the vertex 5 and 8. Consider the vertex 5 as 'x', and the vertex 8 as 'y'.
 - $d(x, y) = d(x) + c(x, y) < d(y)$
 - $= (9 + 15) < 15$
 - $= 24 < 15$
 - Since 24 is not less than 15 so we will not update the value $d(8)$ from 15 to 24.

-
- Now, we consider the vertex 6. First, we calculate the distance between the vertex 5 and 6. Consider the vertex 5 as 'x', and the vertex 6 as 'y'.
 - $d(x, y) = d(x) + c(x, y) < d(y)$
 - $= (9 + 2) < \infty$
 - $= 11 < \infty$
 - Since 11 is less than infinity, we update $d(6)$ from infinity to 11
 - Till now, nodes 0, 1, 4 and 5 have been selected.
 - We will compare the nodes except the selected nodes.
 - The node 6 has the lowest value as compared to other nodes. Therefore, vertex 6 is selected.

-
- Since vertex 6 is selected, we consider all the direct paths from vertex 6. The direct paths from vertex 6 are 6 to 2, 6 to 3, and 6 to
 - 7.
 - First, we consider the vertex 2. Consider the vertex 6 as 'x', and the vertex 2 as 'y'.
 - $d(x, y) = d(x) + c(x, y) < d(y)$
 - $= (11 + 4) < 12$
 - $= 15 < 12$
 - Since 15 is not less than 12, we will not update $d(2)$ from 12 to 15

-
- Now we consider the vertex 3. Consider the vertex 6 as 'x', and the vertex 3 as 'y'.
 - $d(x, y) = d(x) + c(x, y) < d(y)$
 - $= (11 + 14) < \infty$
 - $= 25 < \infty$
 - Since 25 is less than ∞ , so we will update $d(3)$ from ∞ to 25.
 - Now we consider the vertex 7. Consider the vertex 6 as 'x', and the vertex 7 as 'y'.
 - $d(x, y) = d(x) + c(x, y) < d(y)$
 - $= (11 + 10) < \infty$
 - $= 22 < \infty$
 - Since 22 is less than ∞ so, we will update $d(7)$ from ∞ to 22.

-
- Till now, nodes 0, 1, 4, 5, and 6 have been selected. Now we have
 - to compare all the unvisited nodes, i.e., 2, 3, 7, and 8. Since node 2 has the minimum value, i.e., 12 among all the other unvisited nodes. Therefore, node 2 is selected.
 - Since node 2 is selected, so we consider all the direct paths from node 2. The direct paths from node 2 are 2 to 8, 2 to 6, and 2 to 3.
 - First, we consider the vertex 8. Consider the vertex 2 as 'x' and 8 as 'y'.
 - $d(x, y) = d(x) + c(x, y) < d(y)$
 - $= (12 + 2) < 15$
 - $= 14 < 15$
 - Since 14 is less than 15, we will update $d(8)$ from 15 to 14.

-
- Now, we consider the vertex 7. Consider the vertex 3 as 'x' and 7 as 'y'.
 - $d(x, y) = d(x) + c(x, y) < d(y)$
 - $= (19 + 9) < 21$
 - $= 28 < 21$
 - Since 28 is not less than 21, so we will not update $d(7)$ from 28 to 21.

-
- Till now, nodes 0, 1, 2, 4, 5, and 6 have been selected. We compare
 - all the unvisited nodes, i.e., 3, 7, and 8. Among nodes 3, 7, and 8, node 8 has the minimum value.
 - The nodes which are directly connected to node 8 are 2, 4, and 5.
 - Since all the directly connected nodes are selected so we will not consider any node for the updation.
 - The unvisited nodes are 3 and 7. Among the nodes 3 and 7, node 3 has the minimum value, i.e., 19.
 - Therefore, the node 3 is selected.
 - The nodes which are directly connected to the node 3 are 2, 6, and
 - 7. Since the nodes 2 and 6 have been selected so we will consider these two nodes.

-
- Now, we consider the vertex 6. Consider the vertex 2 as 'x' and 6 as 'y'.

$$d(x, y) = d(x) + c(x, y) < d(y)$$

- $= (12 + 4) < 11$

- $= 16 < 11$

- Since 16 is not less than 11 so we will not update $d(6)$ from 11 to 16.
- Now, we consider the vertex 3. Consider the vertex 2 as 'x' and 3 as 'y'.
 - $d(x, y) = d(x) + c(x, y) < d(y)$
 - $= (12 + 7) < 25$
 - $= 19 < 25$
- Since 19 is less than 25, we will update $d(3)$ from 25 to 19.